



System -
Coding |
REC | 3
Feb

Course Progress

Go Back

Back To Practice

Save

Q1

Select Language : C (GCC 9.2.0)

Completed

First Fit Contiguous Memory Allocation

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

1. The first line contains an integer m representing the number of memory blocks.
2. The next line contains m integers representing the sizes of each memory block.
3. The next line contains an integer n representing the number of processes.
4. The next line contains n integers representing the sizes of each process.

```
1 #include <stdio.h>
2
3 int main() {
4     int m, n;
5     printf("Enter number of memory blocks:\n");
6     scanf("%d", &m);
7     int block[m], orig[m];
8     printf("Enter size of each block:\n");
9     for (int i = 0; i < m; i++) {
10         scanf("%d", &block[i]);
11         orig[i] = block[i];
12     }
13
14     printf("Enter number of processes:\n");
15     scanf("%d", &n);
16     int proc[n];
17     printf("Enter size of each process:\n");
18     for (int i = 0; i < n; i++) {
```

Test Cases

Run Sample Cases

Run All Cases

Custom Test Case

Custom Test

Success

Input :

Expected Output :

5
100 500 200 300 600
4
212 417 112 426

Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 2 of size 500 with Fragment 288



System -
Coding |
REC | 3
Feb

Course Progress

Go Back

Back To Practice

Save

Q1

Select Language : C (GCC 9.2.0)

Completed

Best Fit Contiguous Memory Allocation

Write a C program to implement the Best Fit Contiguous Memory Allocation technique. The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size. If no suitable block is available, the process is marked as not allocated. The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

- The first line contains an integer representing the number of memory blocks.
- The second line contains the sizes of the memory blocks.
- The third line contains an integer representing the number of processes.
- The fourth line contains the

```
1 #include <stdio.h>
2
3 int main() {
4     int m, n;
5     printf("Enter number of memory blocks:\n");
6     scanf("%d", &m);
7     int block[m], orig[m];
8     printf("Enter size of each block:\n");
9     for (int i = 0; i < m; i++) { scanf("%d", &block[i]);
10
11     printf("Enter number of processes:\n");
12     scanf("%d", &n);
13     int proc[n];
14     printf("Enter size of each process:\n");
15     for (int i = 0; i < n; i++) scanf("%d", &proc[i]);
16
17     int alloc_block[n], alloc_cur[n];
18     for (int i = 0; i < n; i++) { alloc_block[i] = -1; }
```

Test Cases

Run Sample Cases

Run All Cases

Custom Test Case

Custom Test

Success

Input :

Expected Output :

5
100 500 200 300 600
4
212 417 112 426

Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 4 of size 300 with Fragment 88
Process 2 of size 417 is